Strata — Developer Guide

For engineers implementing tickets

strata-kb v1.34.0 · 2026-11-01

1. Who this guide is for

You write product code. Tickets arrive already grounded in the organisation's knowledge base, with citations pinned to a specific version of a specific document, and you implement them.

Strata does two things for you.

It tells you what the source actually said. Not "check the handbook" — the exact section, at the exact revision the analyst was reading, with a warning if it has changed since.

It publishes what your codebase is, so everyone else — analysts drawing architecture diagrams, other teams, future you — can search your services, dependencies, endpoints and schema without reading your repository.

1.1 What a dev repo does and does not do

Does	Does not
Read the hub's published knowledge, including the BA's published tickets	Ingest documents from outside the repo
Publish knowledge about **its own code**	Publish domain knowledge
Run a five-command implementation workflow	Merge or ship anything by itself

The boundary is deliberate. A `child` repository brings in outside documents; a `dev` repository ingests nothing and authors only knowledge about itself. That is what makes it safe for your CI to publish continuously.

1.2 The two documents your repo publishes

Document	Origin	Regenerated	Needs review
`<repo_id>-code`	`kb code-ingest` — deterministic, no model	on demand, from the working tree — never committed; CI publishes it when `kb publish` pushes a `kb-publish/*` tag	no
`<repo_id>-svc`	drafted by a model from that evidence, corrected by you	never	always

`-code` is machine fact: service names, technologies, dependencies, commands, endpoints, schema, folder structure. `-svc` is human claim: what each service is actually *responsible for*.

They are joined by section id — `svc.<name>` exists in both — so an analyst drawing a C4 container gets the alias, label and technology from `-code` and the description from `-svc`.

---

2. Setup, once

```
pip install "strata-kb[code]"
kb init --kind dev
```

The `code` extra (`tree-sitter` plus `tree-sitter-language-pack`, about 184 MB) is what lets `kb code-ingest` read routes, pages, jobs and symbols out of source code (§7.5). Without it, routes and symbols (and pages other than the file-system routers) are reported as `not scanned`, jobs stay `partial`, and the SA asks you about them. `kb doctor` warns when it is missing, and `kb-code.yml` installs it in CI.

2.1 Configure

```
# .kb/config.yaml
kind: dev
repo_id: payments
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com
```

- **`hub`** is the only read source for `kb query`, `kb get`, `kb resolve` and MCP.
- **`intake`** is how CI publishes. Ask the hub maintainer to add this repository under `repos:` in the hub's `federation/registry.yaml` — publishing is refused until they do.

If the KB should carry SQLite schema, or extra index tags, say so here — not in `kb-code.yml` — so every local refresh and CI ingest the same way:

```
code_ingest:
 db: [data/app.sqlite]   # explicit only
 tags: [payments]
```

2.2 Connect your assistant

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) were scaffolded already wired to the hub's MCP tools, but through placeholders — `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}` — that nothing has set yet.

Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`; `/kb-auto` is the matching wrapper for full-auto mode, §4.7). It asks for the hub's HTTP base URL and token, writes both into `.env`, makes sure git ignores that file, and then verifies them against the hub, so a wrong URL and a rejected token come back as different errors:

Variable	Example
`STRATA_KB_HUB_URL`	`http://kb-hub.example.com:8321`
`STRATA_KB_HTTP_TOKEN`	the hub's bearer token

A bare re-run verifies again without retyping the token. Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` environment variable over `--token` — that flag leaves the token in your shell history.

`.mcp.json` and `.cursor/mcp.json` read those two variables from your environment, not from `.env` directly, so load `.env` into your shell (`set -a; source .env; set +a`, or use direnv) and restart your assistant — MCP reads the environment only at startup.

2.3 Turn on the PR gate

`kb init` writes `.github/workflows/kb-pr-lint.yml` but cannot enable branch protection for you. Add **`pr-lint` to the branch's required checks**.

> Decide this knowingly. Unlike the BA repo's gate, this one never self-skips. Once required, it blocks **every** pull request lacking the required sections — including bot pull requests such as a dependency bump or a revert.

2.4 Ask whether the hub auto-merges your repo

Auto-merge is a per-repo switch on the hub's Registry page; a hub admin turns it on for your repo. Ask your hub maintainer whether it is on, and know what it does:

> `kb ci-publish` opens **one** hub pull request per push, not one per document. The intake keys the publish branch by repository id and reuses it while a PR is pending — so a push carrying a regenerated `-code` alongside an already-merged `-svc` amend lands **both** in that same PR.

> With the switch on, the hub merges the **whole** PR right after opening it, `-svc` content included. The `-svc` review therefore happens locally, in `/dev-code-seed` step 4, before the section reaches `reviewed` — never at the hub.

If the merge fails, the PR stays open for the hub maintainer and the publish still counts as done. `kb publish` and `kb ci-publish` print "merged" when it merged (the CI step summary still shows `hub PR: <url>`). With the switch on, the hub does not lag main: without it, every push adds to one pending hub PR that someone has to merge. `kb doctor` reports when the hub's `-code` lags.

---

3. Onboarding an existing project

Your repository already has services doing real work. Adopting Strata should not mean documenting the system from a blank page, so there is a one-time seed.

```
/dev-code-seed
```

Seven steps:

#	Step	Who
1	**Preflight** — confirms `hub:`, `repo_id:` and `intake:` are set and the repo is allowlisted on the hub; warns on a dirty tree	agent
2	**Extract** — `kb code-ingest --scaffold-svc` regenerates `-code` and creates a `pending svc:<name>` scaffold per detected service, backed by deterministic code evidence	agent
3	**Draft** — `kb summarize <repo_id>-svc` writes a responsibility paragraph for each from that evidence	agent
4	**Review** — walk every drafted section against its evidence and correct it	**you**
5	**Approve** — `kb approve <repo_id>-svc` flips corrected sections to `reviewed`	you
6	**Flows** (optional) — add `flow.<name>` sections for business flows crossing several services	you
7	**Validate and publish** — `kb build` without `--allow-pending`, then `kb publish --pr`	you

3.1 Step 4 is the actual work

> **The model's draft is a draft, not a fact.** It can name the wrong flow, miss a responsibility, or over-claim one. Approving a section you have not read defeats the entire point of the gate.

Read each `svc.*` section's summary next to its code evidence and correct it. Where a responsibility touches a domain standard, cite `doc-id §section` from the hub instead of restating the rule in your own words.

Budget 10–15 minutes per service. That is the honest cost, and it is a one-time cost. After the seed, `-code` regenerates itself and `-svc` accrues a few lines at a time.

For step 6, skip freely. A missing flow beats a guessed one.

3.2 Why the first publish is `--pr` by hand

The seed happens *before* anything has merged to the default branch, so there is no push for `kb-code.yml` to react to.

Use `kb publish --pr`. Plain `kb publish` would tag the commit and poll the intake for up to ten minutes waiting on a tag-triggered CI run this repository does not have, then fail with a misleading error.

That hub pull request is reviewed by an analyst or architect, like any other hub content — unless the hub has auto-merge switched on for your repo, in which case the hub merges it on the repo's next intake publish (`kb publish --pr` itself does not go through intake); the `-svc` review you did in step 4 is the only one.

---

4. Implementing a ticket

Five commands, scaffolded for Claude Code, GitHub Copilot and Cursor.

```
/dev-implement-ticket <ticket>     intake · check · resolve · ground · placeholders
        │
        ├─▶ dev-design       ── A1: independent reviewer ── GATE 1: you approve the design
        ├─▶ dev-plan         ── A2: independent reviewer ── GATE 2: you approve the plan
        ├─▶ dev-execute         A3 per task · A4 at branch end  (repeatable, resumable)
        └─▶ dev-handover     ── A5: merge-risk review ── GATE 3: you open the PR
                                GATE 4: you merge it
```

`/dev-implement-ticket` is also the orchestrator: given a fresh ticket it runs the first five steps, then invokes the phases in order, detecting how far a ticket already got so it never redoes finished work.

4.1 A ticket id is enough

Give the assistant only a ticket id (`M-<slug>-US<n>`, or a plain ticket stem). Intake gets the ticket **from the hub first**: the BA publishes every ready ticket in the `<repo_id>-plan` document, so the hub is the source of truth.

```
kb ticket fetch M-checkout-US3          # or <repo>:<id> if several repos hold it
```

- It finds the one federation repo whose `*-plan` document has that section. If none does, it exits `1`: the ticket is not published yet, so ask the BA to `kb publish`, or pass a path or paste the body. If several do, it exits `1` and names them; qualify the id as `<repo>:<id>`.
- It writes `docs/impl/<id>/ticket.md` (`--out` to change it) with the title, one provenance line and the ticket body: `> Fetched from hub: <repo>:<doc> §<id> @ <hub rev> — do not edit; amend in the BA repo.`

- It **overwrites** the file on every fetch. The copy is derivable from the hub, so `docs/impl/.gitignore` ignores `*/ticket.md`: never commit it, and never edit it. If the ticket is wrong, the BA amends it in the BA repo.
- An id that is not a plain name (a path, `..`) is refused.

Without the CLI, the assistant reads the section through MCP (`kb_get_section`, `<doc> §<id>` at `l3`) and saves it the same way. A path or a pasted ticket body still works, as the fallback when the hub has no copy or cannot be reached.

The hub copy has **no** `## Definition of Ready`: being on the hub means the BA ticked every box, so no step may stop on its absence.

The parent mission's `## Sequencing` is read the same way when there is no local mission file (`kb get <doc> <M-id> --level l3`), so you can see what the ticket is blocked by without cloning the BA repo.

The grounding check

Right after Intake the flow runs `kb ticket check docs/impl/<id>/ticket.md` (your `<ticket-file>`; skipped with a note when the ticket has no `## Technical grounding`). It reads the hub's `-code` and, for a `[NEW: D<n>]`, the parent mission from the hub's plan when no local `missions/` holds it. The `Grounded on:` revision moves with every commit you merge, so a mismatch alone never fails:

Output	Meaning	What to do
PASS, `[note] grounded @<old>, still valid at @<new> — cited sections unchanged`	Nothing the ticket cites changed	Carry on
PASS, `[warn] … §<id> changed; cited names still hold, review the diff:`	A cited section changed, every cited name is still there	Read the diff and decide; carry on unless it changes the ticket's meaning
PASS, `[warn] some cited facts could not be verified …`	A check was skipped (e.g. a tree cap)	Check those facts by hand
FAIL, `… re-run /sa-ticket-ground`	A cited id, column, route, command, file or compose fact is gone	Stop: a blocker for the BA, whose SA re-grounds the ticket
FAIL, plus `[note] checked against the local -code, not the hub's copy …`	The hub was unreachable or lacks `-code`; your branch may be the cause	Re-run once the hub is reachable and holds the `-code`
FAIL without the re-ground line	An open decision or an undecided `[NEW: D<k>]`	Report it to the BA and stop

What a ticket's status means to you

The hub's **Ticket** menu (`/ui/tickets`) and `kb mission next --from-hub [--repo <ba-repo>]` show every story of a published mission as **done** (its id is in a `hist.*` row of your `-svc`, which `kb svc note` writes at handover), **ready** (its ticket is published), **not ready** (no published ticket yet) or **blocked** (with reasons). Pick a **ready** one with no reason: a **ready** story whose dependency is not done yet shows `US <id> not done` (on the ticket page: "Waiting on: …"), and `kb auto run` skips it until that dependency is done.

On the BA side, `kb mission next` can also report a ticket `draft` with the reason `parent mission rev <M> affects this story`: the mission was amended after the ticket was written, and the BA has not re-authored it. Such a ticket is not published, so you see the story as *not ready* and `kb ticket fetch` finds nothing. Wait for the BA; do not implement from an older copy.

4.2 Pick your entry point

Situation	Run
New ticket, nothing started	`/dev-implement-ticket <ticket>`
Small ticket, the change is obvious	`/dev-implement-ticket <ticket>` — the flow collapses itself; the design is still written, just a bounded one
Design approved, no plan yet	`/dev-plan <id>`
Plan approved, or execution in progress	`/dev-execute <id>`
Code hand-implemented, needs a PR write-up	`/dev-handover <id>`
Lost track of where a ticket stands	`/dev-implement-ticket <id>`
A BA-reported bug (`BUG-<n>`)	`/dev-fix-bug BUG-<n>`
A technical question that needs evidence from this repo and the hub	`/dev-sa <question>`
Just checking a citation	`kb resolve <file>`

Every entry point re-checks citation freshness first. The hub may have published since your last session, so a reference that was `ok` yesterday can be `stale` today. Checking only at handover would be too late.

4.3 Where work lives

File	Written by	Purpose
`docs/impl/<id>/design.md`	`dev-design`	the design
`docs/impl/<id>/plan.md`	`dev-plan`	one task per acceptance criterion, `- [ ]` checkboxes, a `Depends on:` line per task
`docs/impl/<id>/ticket.md`	`kb ticket fetch`	the ticket fetched from the hub; gitignored, overwritten on every fetch, never edited
`docs/impl/<id>/context.md`	`dev-implement-ticket`	resolved-context cache; gitignored, regenerated on demand
`docs/impl/<id>/review/`	`dev-design`, `dev-plan`, `dev-execute`, `dev-handover`	A1/A2 round reports, pre-fix copies and round diffs, per-task diffs and reviewer reports, the branch diff, the merge-risk report; gitignored, derivable from git and the review records below

`kb init --kind dev` and `kb impl tidy` move an old flat layout (`docs/impl/<id>-plan.md`, …) into these folders.

The design and plan files each grow a `## Review record` table as their reviews run — the committed history of every round, not a separate file.

State is derived from the first two files, the current branch, and whether a PR is open — **never from conversation history**. Any phase resumes cold in a brand new session.

`kb init` never edits your `docs/impl/` content: it adds `.gitkeep` and a `.gitignore` so the context cache, the fetched ticket and the review artefacts never land in a pull request, and moves an old flat layout into `<id>/` — never overwriting. Commit the renames.

Waves and lanes. Every task in the plan carries a `Depends on:` line under its `### Task <n>:` heading — `none`, or the tasks whose Interfaces it consumes or whose files it shares. `dev-plan` derives it; the A2 reviewer checks it. `kb plan waves docs/impl/<id>/plan.md` turns those lines into waves (wave n = tasks whose dependencies all sit in earlier waves) and fails when two tasks share a file without a dependency between them, when there is a cycle, or when a line is missing — a plan like that has no safe parallel order. A plan where every task lacks the line predates waves and runs sequentially.

`dev-execute` runs it after isolating the branch. A wave of one task runs as before; a wave of several runs each task in its own git worktree lane cut from the ticket branch, at most 3 lanes at a time, merged back in task order with `--no-ff`, the full suite run once per wave. A merge conflict means the plan was wrong and goes back to `dev-plan`.

A task that rebuilds or drives a shared running environment (an e2e or compose stack, fixed host ports) carries a `Serial resource: <name>` line next to `Depends on:`. It adds no dependency; `kb plan waves` prints such tasks after `serial:` in their wave (`wave 5: task 3; serial: task 14, task 20`), and `dev-execute` keeps them out of the lanes and runs them one at a time in the ticket worktree once the wave's lane tasks are merged and ticked.

4.4 Four human gates, five agent review rounds

Nothing in this pipeline merges or ships without a human.

1. **Design approved** — you sign off before any plan is written.
2. **Plan approved** — you sign off before any code is written.
3. **PR opened** — the agent writes the body; you open it.
4. **Merged** — you review and merge.

The agent does neither of the last two itself.

Before each of the first three gates, a different agent has already read the work back in a context of its own — never the same agent that produced it:

	Where	What it looks at
A1	`dev-design`, before GATE 1	whether the design matches the ticket
A2	`dev-plan`, before GATE 2	whether the plan is test-first and runnable
A3	`dev-execute`, per task	whether this task's diff meets its spec, and whether the code is good
A4	`dev-execute`, after the last task	whether the finished branch rounds out the ticket
A5	`dev-handover`, before GATE 3	whether merging into the default branch is safe

Criteria live in `docs/pr-review-rubric.md`, overridden by `docs/pr-review-rubric.local.md`. A3, A4 and A5 allow at most 3 fix/review passes. They only count as clean once no BLOCKER and no SUGGESTED finding is left open — NOTE and NITS get recorded, not fixed — and a BLOCKER or SUGGESTED still standing after round 3 stops the flow and hands it to you. A1 and A2 follow the shorter rule below.

A1 and A2 stop after round 1 plus one verification round; there is no third round. The author runs a self-check before any reviewer sees the draft. Round 1 must be exhaustive: the reviewer returns a coverage matrix (every task against every question, or every AC and axis item against the design sections) and tags each gap with a defect class. The fix sweeps the whole artefact for every instance of each class, not only the one cited. Later rounds verify the fix against the diff instead of re-reviewing everything. A design over 8 ACs or 600 lines, or a plan over 12 tasks or 600 lines, is reviewed in parallel slices, and a lead reviewer re-checks every slice finding, looks across slice borders, and checks for drift from the ticket and the design.

A new gap that the verification round finds outside the fix diff is recorded as `round-1 coverage miss`. It does not block, unless it is a BLOCKER. A gap that still blocks after the verification round stops the flow and goes to you, with the open classes. That means a BLOCKER that cannot be checked at execution, or a SUGGESTED gap with no row in `## Carried gaps`.

Carried gaps. A design or plan lists leftover gaps in a `## Carried gaps` table below `## Review record`:

```
| Gap | Class | Severity | Raised in | Target task | Status |
|---|---|---|---|---|---|
```

Two kinds of row go in. One is every `execute-verifiable` gap from any round (a gap that running the code or its tests settles; the rubric lists them under `### Execute-verifiable gaps`). The other is every round-2 SUGGESTED gap outside the fix diff. In a design the target task reads `plan` until the plan names one. GATE 1 and GATE 2 are offered once no BLOCKER is open and every open SUGGESTED gap has a target task in the table. An `execute-verifiable` gap never counts toward the gate, whatever its severity, and it never goes to the fix subagent: it is carried unfixed, not fixed on paper.

During `dev-execute`, the implementer of a task receives the carried rows that name it and must resolve them. Spec review checks that it did; the orchestrator sets `Status` to `closed task <n>` when the task is ticked.

Deviations. When the code of a task differs from its task block or from a design section, the implementer changes that plan or design text in the same task commit and adds a row to `## Execution deviations` at the end of `plan.md`:

```
| Task | Doc + section | What changed | Why |
|---|---|---|---|
```

The documents therefore follow the code. In a parallel lane the implementer edits neither file: it returns its rows and doc edits in its report, and the orchestrator applies them at merge-back, before A3. Spec review checks that the text matches the code. A deviation that changes the meaning of an AC or a public contract gets one **delta review** of that doc edit alone, by a fresh `design-reviewer` or `plan-reviewer`, one round, before A3. A BLOCKER from it goes to you; a SUGGESTED finding becomes a carried row.

Where your assistant's runtime cannot dispatch subagents — the default for the Copilot and Cursor wrappers — the review still runs, but as a single pass reading only what it was handed, and it is a **report, not a gate**: the phase stops there and hands the result to you instead of advancing on its own say-so.

4.5 Fixing a bug

A bug the BA published is `BUG-<n>` (`> Type: bug` or `hotfix`, `> Severity: S1`–`S4`). It has no user story and no acceptance criteria: it is done when its steps no longer reproduce the wrong result. `/dev-fix-bug BUG-<n>` runs it in the style of `superpowers:systematic-debugging`:

1. **Intake** — fetch the bug. `kb ticket fetch BUG-<n>` writes `docs/impl/<id>/ticket.md` and copies the bug's screenshots (PNG or WebP) to `docs/impl/<id>/assets/`; it warns once per image missing from the hub clone, and you open `<hub>/assets/<name>`. MCP `kb_ticket_fetch` returns text only. Branch `fix/<BUG-id>`, or `hotfix/<BUG-id>` from the production line you name.
2. **Reproduce** — a failing test that shows the Actual result. No fix without it.
3. **Root cause** — written hypotheses in `docs/impl/<id>/rootcause.md`. Three failed hypotheses stop the flow for an architecture talk with you.
4. **Fix** — one change at the root cause, full suite green.
5. **Handover** — `/dev-handover` with bug rules: no design or plan, and the reproduction test plus the root cause replace them.

4.6 Questions for the BA, and `/dev-sa`

`dev-design` and `dev-plan` do not settle a doubt by guessing. Their author subagents never call the hub; they list each doubt under `## Questions raised` in the draft, tagged `business` or `technical`, with the ACs it blocks. The orchestrating skill sorts them:

- **Technical** (how to build it, where it goes, which component to reuse) goes to `dev-sa`, the solution architect of this repo. It reads the source and the hub's `-code`, `-svc` and domain documents and returns a decision with reasoning, rejected alternatives and `path:line` / `doc §section` evidence. The skill writes it into the draft's `## Technical decisions` as `- SA-<n>: <decision> — why — evidence` and re-dispatches the author. Nothing goes to the hub. You can call it yourself: `/dev-sa <question>` for the ticket in flight; it returns the line for you to record. When it decides the answer would change what an AC means, it returns `business question:` and the question goes to the BA instead.
- **Business** goes to the BA through MCP `kb_question_ask`, and appears in the draft's `## Open questions` as `- Q<id> — <question> — blocks: <AC ids|none>`, where `<question>` is the **Decision needed** sentence on one line. The BA answers with `/ba-answer-questions`.
- **Question format.** The stored question is markdown with fixed English labels; the content is in the ticket's language: `**Decision needed:**` (one sentence), `**Background:**` (2-4 bullets), `**Options:**` (numbered, each with its consequence), `**Dev recommends:**` and `**AC impact:**`. Ask one decision per question, name no internal id (DD12, SA-1, R1-13) unless the text says what it is, and give a `context` that points at a place the BA can open, such as `design.md §9.3`, never a review-round code. The hub shows the question with its line breaks and lists. The Q&A page renders this markdown with raw HTML off.
- **A blocking question stops the gate.** A design or plan with an open question that blocks an AC cannot pass GATE 1 or GATE 2; the skill stops and gives you the hub's Q&A link to paste in chat. A question that blocks no AC does not stop it, and the A1 and A2 reviewers treat it as a known open item. The Pre-code axis of the review rubric accepts a hub-asked `Q<id>` as the way to carry an ambiguity.
- **The `BA-Q` marker.** The `State:` line of the Next-step block gains a fifth marker, `BA-Q: none | open Q<ids> | waiting Q<ids>`.
- **Resuming.** On every entry `/dev-implement-ticket` reads `kb_question_list` for the ticket. An *answered* question you have not marked yet is shown with its changed sections; the flow re-fetches the ticket, marks the `## Open questions` line `— answered rev N — re-draft` (the phase re-dispatches its author once for it, then drops `— re-draft`), and returns to `dev-design` if a changed section touches an approved design (it goes back to `draft`). A *waiting* question (answered, but the BA's new ticket revision is not on the hub yet) is information only: do not act on it. An *open* blocking one stops the flow.
- **Shared hub server only.** `kb_question_ask`, `kb_question_list` and `kb_question_answer` work only when MCP points at the hub's HTTP server (`kb mcp-setup`); a hub URL read through a local cache clone, or any other target, answers `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`. Any authenticated caller may write and the names are self-declared (`git config user.name`), so they track who asked and nothing more. At most 50 questions per ticket can be open. When Q&A is unavailable and every

`Q<id>` line in the design or plan carries `– answered rev N` (or there are none), `/dev-implement-ticket` continues with `BA-Q: unknown (Q&A unavailable)` and a note; if any `Q<id>` line lacks it, it stops, because it cannot know the state. In `dev-design` and `dev-plan` a business question with `blocks: none` then falls back to `OPEN(BA)`; only a blocking one stops.

4.7 Full-auto mode (optional)

A switch that lets this Dev repo run on its own, beside a BA repo in full-auto mode, until every ticket on the hub is done: no gate waits for you. The agent approves GATE 1 and GATE 2 after the independent review, opens the PR, fixes CI until it is green, merges, and runs `kb publish`. Every decision is a line in `docs/impl/<id>/auto-decisions.md`, copied into the PR's `## Notes`. Built for **one BA repo and one Dev repo per hub**. Full reference: README, "Full-auto mode".

Security. Read this first. The permission deny list is defence in depth, not a boundary. The run merges with **admin rights** (only when review is the one missing requirement and every check is green) and nobody reviews the PR. A ticket misread by the agent goes straight to `main` and the hub. The repo owner must enable GitHub branch protection on `main` (no force-push, no deletion, **include administrators**) and keep `kb-test.yml` (or the workflow carrying the `kb-test` marker) as a **required check** (branch protection picks it by job name: `test-<lang>`). Do **not** require approving reviews on `main`: the agent cannot approve its own PR, and `gh pr merge --admin` is refused when administrators are included. If you do require reviews, exclude administrators from that rule and accept that the agent can then bypass it.

Set up once.

1. A mission and its tickets are on the hub (the BA side, by hand or in auto mode). The hub must be 1.29.0 or later, because the runner polls Q&A over `GET /api/questions`.
2. `intake:` is set in `.kb/config.yaml` and the hub's Registry page has auto-merge **on** for this repo; without it no `hist` row lands and no story reaches `done`.
3. Re-run `kb init --kind dev`. It merges the permission allowlist, with the test, install, lint and build commands of your `langs:`, into `.claude/settings.json`, and refreshes the skills (Auto mode sections) and adds `/dev-test-workflow`.
4. Export `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN`; have `claude` logged in with your subscription.
5. After `kb auto on`, commit and push `.kb/config.yaml` and `.claude/settings.json` to the default branch. `kb auto run` refuses until you do: `kb publish` refuses a dirty `.kb/`, so no ticket would reach `done`.

Run. `kb auto on`, then commit and push those two files, then `kb auto run` — or `/kb-auto run` from your editor, which does all three and starts the loop detached (log in `.kb/auto/run.log`). `kb auto status` shows the task, attempts, the questions it waits on and the stop reason; `kb auto off` stops after the current task.

What it does. One fresh `claude -p` process per ticket. First, if no workflow runs the repo's tests, `/dev-test-workflow` writes and merges `kb-test.yml`, so "green CI" means tests ran. Then bugs first (`/dev-fix-bug`; a hotfix branches from `main` unless `auto.hotfix_base` names a production branch), then the first `ready` ticket that has no mission and is not a bug (`/dev-implement-ticket`), then the first `ready` story in mission order whose dependencies are `done` (`/dev-implement-ticket`). When a business question blocks a ticket, the Dev asks the BA through hub Q&A and waits; once every question is `answered` the same ticket resumes in a fresh process.

When it stops. After two failed attempts at one ticket (a red CI after two fix rounds stops it at once); fix it, then `kb auto run` resumes it. A tool outside the allowlist ends the task `failed command-denied <command>`: add that command to `permissions.allow` in `.claude/settings.json` by hand. A blocked `kb` command that has a `strata-kb` MCP tool (denied, missing, refused here, or unable to reach the hub) counts as no CLI: the skill runs the MCP tool instead of failing. A ticket that ended `done` but is still picked after 5 checks is skipped (`skipped` in `kb auto status`) and the loop moves on; when every ticket left is skipped it stops, and `kb auto run` retries them. A

usage-limit wait (15, 30, 60 minutes) is not a failure. It finishes, exit `0`, when every story and every unparented ticket and bug is `done`. The default branch must be `main`.

4.8 Several Devs on one repo

Two Devs who pick the same hub ticket would build it twice. A **ticket claim** on the hub prevents that. `/dev-implement-ticket` and `/dev-fix-bug` run `kb claim take <id>` in Intake, and every later phase refreshes the claim.

- **One holder per ticket.** The claim names its holder (`<git user.name>@<hostname>`, or `STRATA_KB_CLAIM_AS` / `--as`) and its phase: `claimed`, `design`, `plan`, `execute`, `handover`, `review`, `publishing`, `waiting-qa`. A second Dev who runs the same ticket gets `taken by <holder> (<phase>, updated <x> ago)` (exit `3`) and the skill stops: pick another ticket, or ask a hub admin to release it. A BA repo has no claims, and `dev-test-workflow` carries none.
- **It expires after 4 hours without an update.** The hub env `STRATA_KB_CLAIM_TTL` (seconds, default `14400`) sets it. `dev-design`, `dev-plan`, `dev-execute` and `dev-handover` update the claim on entry, `dev-execute` after each task commit, and `kb auto run` sends a heartbeat every `auto.claim_heartbeat` seconds (default `900`, in `.kb/config.yaml`).
- **Held through review and publishing.** Once you have opened the PR, tell `dev-handover` (or re-run `/dev-handover <id>`): it finds the PR and sets `review` with its URL; until then the claim expires after 4 hours. A claim in `review` or `publishing` with a PR never expires, so the ticket stays yours while the PR waits. A claim the auto runner marked `stopped` (the run stopped with a reason) never expires either, and resumes when you run it again.
- **Taking over.** After an expiry or a release, the next Dev who claims the ticket and finds its branch on `origin` (`auto/<id>`, `<id>`, `fix/<BUG-id>`, `hotfix/<BUG-id>`) continues on that branch and logs whose work it took over. If your claim was released while you work, the next update exits `3` (`lost: <reason> by <who>`) and the skill stops without pushing.
- **Look and release.** `kb claim list [--json]` shows what is held; `kb claim release <ticket> [--reason TEXT]` gives yours back. A hub admin can release anyone's claim with the **Release** button (and a reason) on the ticket page of the Ticket menu. The menu shows the claim as *In progress*, *In review*, *Publishing* or *Stopped:* .
- **Hub first.** The commands need `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` and a hub that is 1.31.0 or later. An older hub has no `/api/claims`: `kb claim` exits `1` with `hub server has no /api/claims — upgrade it to strata-kb 1.31.0 or later`, and `kb auto run` stops. Upgrade the hub, then the CLI, then re-run `kb init --kind dev`.
- **The publish guard.** In a dev repo `kb publish` refuses to run from a checkout that is behind `origin/<default>` (`behind origin/<default> — pull, then kb publish`; also `no origin remote — …` and `could not fetch origin/<default>: …`), because the snapshot would drop another machine's newer `hist` row. A squash merge leaves the ticket branch behind, so after the merge switch to the default branch, pull, then publish: `git fetch origin && git switch <default> && git pull --ff-only origin <default>`, then `kb publish`. `dev-handover` says the same.

---

5. What is enforced, and what is only asked

Be clear-eyed about the difference.

5.1 Machine-enforced



The PR body is short: `## Ticket` (what was asked), `## Changes` (what this PR did), `## Testing` (AC→test line, the pasted test command with its summary lines, A5's `Blocking: No`), and an optional `## Notes` (out of scope, not handled, findings). `kb pr lint` fails when Ticket, Changes or Testing is missing or still holds the template's comment, when Testing has no pasted output or no fence containing the plan's test command, or when its `Blocking:` line is missing or reads `Yes`.

5.2 Prompt-only — stated in every wrapper, measured by nothing

These are rules the skills repeat. Nothing in `kb` enforces or measures them. The only check is the human at the gate — which means you.

- **TDD.** No production code without a failing test observed first, at every step of `dev-execute`. Exempt classes are named in `docs/tdd-exemptions.md` and declared in the plan.
- **Shown verification.** No completion claim without the real command output. `kb pr lint` checks the PR body, not the session.
- **Pinned values, verbatim.** Every standard-derived value comes from the resolved section at its pinned hub version, with a citation comment.
- **The ticket is read-only.** Findings go back to the analyst; the agent never edits the ticket.
- **Gates 1 and 2.** Design and plan approved before the next phase. The `status:` header records it; a human grants it.
- **`OPEN(BA)`.** An ambiguous acceptance criterion is escalated, never reinterpreted. During design and plan the escalation is a hub question (`Q<id>`, §4.6); `OPEN(BA)` stays for what `dev-execute` meets mid-implementation, which returns to `dev-design`.
- **A1–A4.** The four agent reviews before GATE 3 are prompt discipline. Nothing in `kb` measures whether they ran. Only A5 has teeth: `kb pr lint` reads the `Blocking:` line in the PR's `## Testing` section. An agent that skips A1–A4 and writes an honest `Blocking: No` still passes — the gate sits where the merge actually happens, not at every step before it.

---

6. Working with citations

```
kb resolve tickets/ABC-123.md    # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>
kb get hr-handbook 4.12 --level l3
```

Status	Meaning	What to do
`ok`	The pinned content still matches what is published	Implement against it
`stale`	The source was amended after the ticket was written	Run `kb diff`, then ask the analyst whether the acceptance criteria still hold. **Do not silently implement the new version.**
`broken`	The reference no longer resolves	Escalate. The ticket is not implementable as written.

When a value in your code comes from a standard — a format, an enum, a threshold, a field length — take it from the resolved section at the pinned version and leave a citation comment next to it. Six months later that comment is the only thing connecting the constant to its justification.

---

7. `kb code-ingest` — what it extracts

```
kb code-ingest                          # defaults are usually right
kb code-ingest --db data/app.sqlite     # name a database explicitly
kb code-ingest --json                   # machine-readable report
```

Eleven extractors, organised by **artefact kind rather than programming language**, because Strata is adopted across many project lines. The first seven read build, deploy and contract files; the last four read source code with tree-sitter (§7.5).

Extractor	Section prefix	Reads
`services`	`svc.<name>`	compose files, Dockerfiles, k8s manifests, solution and workspace files
`deps`	`dep.<ecosystem>`	manifests for Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift, Dart
`commands`	`cmd.<purpose>`	package scripts, Makefiles, tox, shell scripts, CI run steps
`tree`	`struct.tree`	the files Git tracks, so an ignored virtualenv never appears
`schema`	`db.<table>`	SQL migrations, Prisma, Alembic, EF, and explicitly named SQLite files
`integrations`	`int.<name>`	environment-variable **keys** only
`api`	`api.<tag>`	in-repo OpenAPI and Swagger contracts
`routes`	`route.<slug>`	HTTP routes declared in source code (needs `[code]`)
`pages`	`page.<slug>`	UI page routes (file-system routers work without `[code]`)
`jobs`	`job.<slug>`	scheduled jobs of framework schedulers (needs `[code]`)
`symbols`	`sym.<slug>`	exported names, enum and object members, CSS custom properties (needs `[code]`)

`tree` also writes `struct.files.<dir>`, and every extractor reports into `struct.coverage` (§7.5). `tree` always finds something and the four source-reading extractors do not count as evidence of a codebase, so the zero-detection rule is: `kb code-ingest` exits `1` when **none of `services`, `deps`, `commands`, `schema`, `integrations` and `api`** found anything. A document holding only a folder listing and source-derived sections is a misconfiguration, not knowledge.

7.1 Two security properties

These are design properties, not conveniences.

1. **Keys, never values.** The only environment file the integrations extractor opens is an `.env.example`, `.sample` or `.template` — never a real `.env` — and its pattern has no capture group around the value at all, so there is nothing to leak whatever a matched line contains. Compose `environment:` blocks *are* read and do hold real inline values, but only the key survives.
2. **Explicit database input.** SQLite schema is read only from paths you name on the command line. A scratch or fixture database in the repository can never become published company knowledge by accident.

7.2 Stated parser limits

These are limits, not bugs.

- `schema` replays `CREATE TABLE`, `ALTER TABLE … ADD COLUMN` and the index and key statements below in sorted filename order. It does not implement a SQL dialect.
- A `CREATE TABLE` for a name already created in a *different* migration directory is dropped with a warning naming both files, and an `ADD COLUMN` applies only inside the directory of its `CREATE TABLE`. Two independent schemas sharing a table name are never merged into one that exists nowhere.
- Partition clauses, primary keys, `CREATE INDEX` and `ADD CONSTRAINT … PRIMARY KEY|UNIQUE` are read. A statement `schema` does not apply (`RENAME`, an unknown `ALTER` action) is a site: the `schema` coverage row of that scope and reader reads `partial` and names the `file:line`. `FOREIGN KEY` and `CHECK` constraints only warn.
- A table name outside the recognised identifier shape is skipped **with a warning naming the file**. The reader counts `CREATE TABLE` keywords and compares against what it recognised, so a skip is never silent.
- EF Core migrations are recognised by table name only.
- `build.gradle` is matched by regex, not parsed as a DSL.
- Maven `<plugins>`, `setup.cfg` `extras_require`, `pnpm-workspace.yaml` and compose `healthcheck:` are not read.

7.3 A destination it did not write is refused

If `.kb/<doc_id>/` already holds Markdown or a manifest and that manifest is missing, unreadable, not titled `… — code knowledge`, or lists a section id outside the generated prefixes, `kb code-ingest` exits 1 and writes nothing. There is no override flag — move the document or choose another `--doc-id`.

7.4 Determinism and the dirty-tree caveat

Re-running on the same commit on the same platform produces byte-identical files, which is what makes the CI publish a true no-op on an unchanged tree.

Two honest limits: a few glob matches case-normalise per operating system, so the guarantee is per-platform (CI is always Linux, so what gets published is stable); and the manifest's revision derives from the **HEAD commit** while your working tree may hold uncommitted changes. `kb code-ingest` detects that and prints a `dirty_tree` warning.

Local `-code` is what CI would produce if your tracked files were committed now; `kb doctor` warns when your installed strata-kb differs from the version `kb-code.yml` pins.

7.5 Routes, pages, jobs, symbols and coverage

Four extractors read source syntax with tree-sitter and need `pip install "strata-kb[code]"`. The pin is exact (`tree-sitter-language-pack` 0.13.0, never 1.x, which downloads grammars from the network at run time), so the same commit gives the same bytes on every machine. Languages: JS/TS, Java/Kotlin, C#, Python, PHP, Go, Rust, Swift and Dart, plus CSS/SCSS custom properties.

Section	Holds
`route.<slug>`	One per controller class, router file or route file. Table `Method | Path | Handler | File | Request type`, with class, router, group and global prefixes joined; L3 adds the fields of a request type defined in the repo
`page.<slug>`	One per front-end app. Table `Path | File | Layout/group`; route groups such as `(admin)` and dynamic segments (`[id]`, `:id`, `[...slug]`) are kept as written; a Next.js `middleware` matcher is one `Middleware matcher` row
`job.<slug>`	One per declaring file or manifest. Table `Name | Schedule | Handler | File`; the schedule is the expression as written, `hosted`, or `(dynamic)`
`sym.<slug>`	One per package root (a directory with its own manifest), else per top-level directory. Table `Name | Kind | File`; kinds `function`, `class`, `interface`, `type`, `const`, `enum`, `member`, `css-var`
`struct.files.<dir>`	Every tracked file under one top-level directory, uncapped; `struct.tree` stays the capped overview
`db.<table>`	Gains `Table`, `Primary key`, `Partition` and `Indexes` rows. A Prisma model and the SQL migration of the same table merge into one `db.<Model>` section; a disagreement shows both values with their source

Supported frameworks:

- **Routes:** NestJS, Express, Next.js, Nuxt, Spring Boot, Quarkus, Micronaut, ASP.NET Core, FastAPI, Flask, Django, Gin, Laravel, Symfony, Actix Web, Axum, Rocket, Vapor.
- **Pages:** Next.js, Nuxt, SvelteKit, Angular, Vue, React (react-router), Flutter (go_router).
- **Jobs:** NestJS, Spring / Quarkus / Micronaut, Hangfire, Quartz and .NET hosted services, Celery, Laravel, Symfony, node-cron, BullMQ, robfig/cron, tokio-cron-scheduler, Kubernetes `CronJob`.

Sections split by file, package or top-level directory, never by size, and no list is truncated. Only names are emitted; the literal strings are route and page paths, middleware matchers, cron or interval expressions and CSS custom-property names. Env values, constant values and enum member values never are.

`struct.coverage` tells the SA how far to trust an absence. One table, `Area | Scope | Framework/language | Status | Detail`, areas `routes`, `pages`, `jobs`, `symbols`, `files` and `schema`:

Status	Meaning
`complete`	Every recognised form in that scope was resolved
`partial`	Something was seen but not resolved: a dynamic route path, a file that does not parse, a re-export the reader cannot follow. Each site is named as `file:line`
`not scanned`	Nothing was looked at; the detail says why (extra missing, a language or framework without an adapter)

A dynamic path in your code therefore makes its scope `partial`, never silently drops the route. **`jobs` is never `complete`:** a job can live where no adapter looks (`pg_cron` in a migration, a hand-written `setInterval`, a crontab), so its row is always `partial` with the detail `jobs outside framework schedulers are not detectable`. A job that exists is listed; an absent one is always a question for you.

What it changes for you: for a route, page, symbol, file or a table's partition, index or primary key, the SA writes a new thing as new only when that scope is `complete`: `<thing> in <path> [NEW: D<n>]` for a route, page or symbol, `<index|partition|primary key> on db.<table> in <path> [NEW: D<n>]` as a `Tables:` sub-bullet for a table's index, partition or primary key (`<path>` is the migration), and `<path> [NEW: D<n>]` as its own `Files:` entry for a file. The scope is the longest one listed in a non-`files` `struct.coverage` row that prefixes the path, and it needs its own row for the area, else the area is `not scanned` there. Otherwise it is an `Open decisions` item, quoting the status. `kb ticket check` enforces it (see the BA guide §3.8). A `-code` made by an older release has no `struct.coverage`, so those areas stay questions for you until its repo is re-published.

Upgrading to 1.28.0: `pip install "strata-kb[code]"` locally; re-run `kb init --kind dev` for the new `kb-code.yml` (it installs the extra in CI); then run `kb publish` to re-publish `-code` (a push to the default branch publishes nothing). Re-run `kb init --kind ba` in BA repos for the updated SA prompt and ticket template.

---

8. Keeping knowledge current

Neither document needs new per-ticket discipline from you.

8.1 `-code` needs nothing

`-code` is never committed. `.gitignore` carries `.kb/*-code/`, and the document is regenerated from your working tree whenever a command needs it: `kb svc note`, `kb build`, `kb doctor` and `kb publish` refresh it first when it is missing, older than `HEAD`, or your tracked files have changes. A new file counts once it is tracked (`git add`). `kb ticket check` reads the hub copy first — that is what the SA grounded on (§4.1).

`kb doctor`'s published-snapshot comparison leaves `-code` out entirely (dev repos never match the hub on it); `kb publish` is what actually refreshes it and sends it to the hub.

`kb-code.yml` re-runs `kb code-ingest` → `kb build` → `kb ci-publish` when a `kb-publish/*` tag is pushed, which is what `kb publish` does. A push to the default branch publishes nothing (the intake accepts a publish only from that tag), so run `kb publish` after a merge; the hub then reflects that commit once its PR merges.



`-code` has one producer, `kb code-ingest`: `kb summarize` refuses it. A hub copy that was summarized before 1.4.0 returns to the deterministic L1/L2 on the next CI publish — a one-time diff, not lost information; L3 is unchanged.

`--scaffold-svc` is deliberately never passed in CI. CI must never create `pending` content, since that would fail its own build step.

8.2 `-svc` accrues at handover

`dev-handover` runs, for every service a ticket touched:

```
kb svc note payments-api --ticket ABC-123 \
 --title "Add refund endpoint" --refs "payments-spec §4.2"
```

That appends one row to `hist.<service>`. It is deterministic and **idempotent**: re-running for the same ticket and service updates the row rather than duplicating it.

You never hand-edit `hist.*`. It is an append-only log written only by `kb svc note`. Those sections are facts — "this ticket touched this service" — not claims, so they need no review and never block a build.

`kb svc note` exits 1 on any of: no resolvable repo id; a missing `-svc` or paired `-code` document; an unknown service (a typo must not invent one); an unreadable manifest; or a manifest/file desync.

8.3 Drift in responsibility text

Responsibility text can drift, and the system **self-reports rather than self-heals**.

When `kb code-ingest --scaffold-svc` refreshes a service's code evidence and that section's summary is already `reviewed`, it compares the old and new rendered evidence as a whole, whitespace-trimmed string — not a semantic diff over specific fields. **Any** change beyond leading and trailing whitespace flags `stale-risk: svc.<name>` in the report.

A real change trips it — a file or table added or removed. So does a change in nothing but *how* the evidence is rendered, with no code changed underneath. It is not an error, and it never touches the summary. It is a flag that a human should look.

---

9. Troubleshooting



The hub-lag check itself runs a full `code-ingest` in a temporary worktree on every `kb doctor` — expect it to add real time to the command. On Windows, the same per-platform glob-casing difference noted in §7.4 can make an otherwise-unchanged repo compare as `lags`; a lag verdict that disappears on Linux (CI) is that caveat, not real drift.

---

10. Command summary

Command	Purpose	Exit
`kb mcp-setup [--hub-url URL] [--no-verify]`	Write the hub's HTTP MCP credentials into `.env` and verify them	`0` ok, `1` no value/probe failed
`kb code-ingest [--db p] [--tags t] [--scaffold-svc] [--json]`	Extract code structure into `-code` (`--db`/`--tags` replace `code_ingest:` for that run)	`0` ok, `1` nothing detected or destination refused
`kb svc note <svc> --ticket <id> --title "…" [--no-refresh]`	Append a row to `hist.<svc>`	`0` ok, `1` unknown service or missing document
`kb build [--strict] [--no-refresh]`	Validate the store	`0` ok, `1` error
`kb doctor [--no-refresh]`	Check KB health (dev repos: refresh `-code` first)	`0` ok, `1` error, `2` stale citation
`kb approve <doc> [--section <id>]`	Flip corrected sections to `reviewed`	`0` ok
`kb publish --pr`	Open a hub PR (the seed's first publish)	`0` ok
`kb ticket fetch <id> [--out p] [--hub u]`	Fetch a ticket from the hub into `docs/impl/<id>/ticket.md`	`0` ok, `1` not on the hub, ambiguous, or unsafe id
`kb ticket check <file> [--hub u]`	The ticket's grounding against the hub's `-code`; a moved revision fails only when a cited fact is gone	`0` PASS, `1` FAIL
`kb mission next --from-hub [--repo id]`	Status of every published story, from the hub alone	`0`
`kb resolve <file>`	Check a ticket's citations	`0` ok, `1` broken, `2` stale
`kb diff <doc> --against <rev>`	What an amendment changed	`0`
`kb query <text>` / `kb get <doc> <section>`	Search and fetch	`0`
`kb plan waves <plan-file> [--json]`	Dependency waves of a `docs/impl/<id>/plan.md`	`0` ok, `1` shared file without a dependency, cycle, unknown task, missing `Depends on:`, or empty `Serial resource:`
`kb impl tidy [--impl-dir]`	Move an old flat `docs/impl/<id>-*` layout into `docs/impl/<id>/`; idempotent, never overwrites	`0` after a report, `1` when the dir is missing
`kb pr lint <file>`	PR evidence gate	`0` PASS, `1` FAIL
`kb auto on|off|status|run`	Full-auto mode: flip the `auto:` switch, show the runner's state, run the loop (`run [--once] [--model] [--effort]`)	`0` ok or done, `2` stopped, `1` refused or error
`kb claim take|update|release|list`	Ticket claims on the hub: one Dev per ticket, `4 h` expiry, held through review and publishing	`0` ok, `1` error, `3` taken or lost

Slash commands: `/dev-implement-ticket`, `/dev-design`, `/dev-plan`, `/dev-execute`, `/dev-handover`, `/dev-code-seed`, `/dev-fix-bug`, `/dev-sa`, `/dev-test-workflow` (full-auto mode).

---

11. Reserved names

`<repo_id>-code` and `<repo_id>-svc` are reserved for this repository's own code knowledge. A domain document must never take either suffix.

Section prefixes are a published contract that analyst tooling and future tooling key on. Each has exactly one owner:

Prefix	Document	Owner
`struct.tree`, `struct.files.<dir>`, `struct.coverage`, `svc.<name>`, `db.<table>`, `dep.<eco>`, `int.<name>`, `api.<tag>`, `route.<slug>`, `page.<slug>`, `job.<slug>`, `sym.<slug>`, `cmd.<purpose>`	`-code`	extractor
`svc.<name>`	`-svc`	you (model-drafted)
`flow.<name>`	`-svc`	you
`hist.<name>`	`-svc`	`kb svc note` only

`svc.<name>` collides across the two documents **on purpose**. It is the join key that lets a C4 `Container(alias, label, technology, description)` be filled from `-code` (the first three) and `-svc` (the fourth).